```
# ======================================================================
# EXPERIMENT 1: Single Layer Feedforward Neural Network from Scratch
# QUESTION/OBJECTIVE: How do you implement a basic feedforward neural
# network mathematically without using deep learning libraries?
# ======================================================================
import numpy as np

# 1. Define dummy data (3 samples, 4 features)
X = np.array([[0.1, 0.2, 0.3, 0.4],
              [0.5, 0.6, 0.7, 0.8],
              [0.9, 0.1, 0.2, 0.3]])

# 2. Initialize weights and bias
np.random.seed(42)
weights = np.random.randn(4, 1) # 4 inputs, 1 output
bias = np.zeros((1, 1))

# 3. Activation function (Sigmoid)
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

# 4. Forward Pass
z = np.dot(X, weights) + bias
output = sigmoid(z)

print("Network Output:\n", output)
```

```
Network Output:
 [[0.69541044]
 [0.86261774]
 [0.73490695]]
```

```
# ======================================================================
# EXPERIMENT 2: Gradient Descent and Backpropagation
# QUESTION/OBJECTIVE: How do you apply gradient descent and the chain
# rule (backpropagation) to train a network and update its weights?
# ======================================================================
import numpy as np

X = np.array([[0.5, 0.1]]) # 1 sample, 2 features
y_true = np.array([[1.0]]) # Target output
weights = np.array([[0.2], [-0.5]])
bias = 0.0
learning_rate = 0.1

def sigmoid(x): return 1 / (1 + np.exp(-x))
def sigmoid_derivative(x): return x * (1 - x) # Assumes x is already sigmoided

# Forward pass
z = np.dot(X, weights) + bias
y_pred = sigmoid(z)

# Calculate Loss (Mean Squared Error)
loss = np.mean((y_pred - y_true)**2)

# Backpropagation (Gradient computation)
error = y_pred - y_true
d_pred = error * sigmoid_derivative(y_pred)

# Gradients
d_weights = np.dot(X.T, d_pred)
d_bias = np.sum(d_pred)

# Gradient Descent Update
weights -= learning_rate * d_weights
bias -= learning_rate * d_bias

print(f"Loss: {loss:.4f}")
print(f"Updated Weights:\n{weights}")
```

```
Loss: 0.2377
Updated Weights:
[[ 0.20608998]
 [-0.498782  ]]
```

```
# ======================================================================
# EXPERIMENT 3: Implementation of Optimizers (SGD, RMSprop, Adam)
# QUESTION/OBJECTIVE: How do you implement, analyze, and compare
# different optimization techniques for network training in PyTorch?
# ======================================================================
```

```
import torch
import torch.nn as nn
import torch.optim as optim

# Simple linear model
model = nn.Linear(10, 2)
dummy_input = torch.randn(5, 10)
dummy_target = torch.randn(5, 2)
criterion = nn.MSELoss()

# Define Optimizers
optimizers = {
    "SGD": optim.SGD(model.parameters(), lr=0.01),
    "RMSprop": optim.RMSprop(model.parameters(), lr=0.01),
    "Adam": optim.Adam(model.parameters(), lr=0.01)
}

# Test each optimizer (resetting model conceptually)
for name, optimizer in optimizers.items():
    optimizer.zero_grad()            # Clear old gradients
    output = model(dummy_input)      # Forward pass
    loss = criterion(output, dummy_target) # Calculate loss
    loss.backward()                  # Backpropagate
    optimizer.step()                 # Update weights

    print(f"{name} optimizer stepped successfully. Loss: {loss.item():.4f}")
```

```
SGD optimizer stepped successfully. Loss: 2.6687
RMSprop optimizer stepped successfully. Loss: 2.5749
Adam optimizer stepped successfully. Loss: 1.7598
```

```
# ======================================================================
# EXPERIMENT 4: CNN Architectures (LeNet, AlexNet, VGG, ResNet)
# QUESTION/OBJECTIVE: How do you implement and structure Convolutional
# Neural Network architectures for image classification?
# ======================================================================
import torch
import torch.nn as nn

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        # Convolutional layers
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.relu = nn.ReLU()
        # Fully connected layer (assuming 32x32 input image)
        self.fc = nn.Linear(32 * 8 * 8, 10) # 10 output classes

    def forward(self, x):
        x = self.pool(self.relu(self.conv1(x)))
        x = self.pool(self.relu(self.conv2(x)))
        x = x.view(x.size(0), -1) # Flatten
        x = self.fc(x)
        return x

model = SimpleCNN()
dummy_image = torch.randn(1, 3, 32, 32) # Batch=1, Channels=3, 32x32 pixels
output = model(dummy_image)
print("CNN Output Shape:", output.shape) # Should be [1, 10]
```

```
CNN Output Shape: torch.Size([1, 10])
```

```
# ======================================================================
# EXPERIMENT 5: Regularization Techniques (Dropout and BatchNorm)
# QUESTION/OBJECTIVE: How do you apply regularization techniques to
# prevent overfitting and improve model generalization?
# ======================================================================
import torch
import torch.nn as nn

class RegularizedNetwork(nn.Module):
    def __init__(self):
        super(RegularizedNetwork, self).__init__()
        self.fc1 = nn.Linear(100, 50)
        self.batch_norm = nn.BatchNorm1d(50) # Batch Normalization
        self.dropout = nn.Dropout(p=0.5)     # 50% Dropout
        self.fc2 = nn.Linear(50, 10)
        self.relu = nn.ReLU()

    def forward(self, x):
```

```
        x = self.fc1(x)
        x = self.batch_norm(x)
        x = self.relu(x)
        x = self.dropout(x) # Apply dropout before next layer
        x = self.fc2(x)
        return x

model = RegularizedNetwork()
model.train() # Set to training mode (important for Dropout/BatchNorm)
dummy_data = torch.randn(16, 100) # Batch of 16
output = model(dummy_data)
print("Regularized Model Output Shape:", output.shape)
```

```
Regularized Model Output Shape: torch.Size([16, 10])
```

```
# ============================================================================
# EXPERIMENT 6: Recurrent Neural Networks (LSTM and GRU)
# QUESTION/OBJECTIVE: How do you implement RNN-based models to process
# sequential data and solve long-term dependency issues?
# ============================================================================
import torch
import torch.nn as nn

# Parameters: input_size, hidden_size, num_layers
lstm = nn.LSTM(input_size=10, hidden_size=20, num_layers=2, batch_first=True)

# Dummy sequence: Batch=3, Sequence Length=5, Features=10
sequence_data = torch.randn(3, 5, 10)

# Forward pass (returns output and final hidden states)
output, (hidden_state, cell_state) = lstm(sequence_data)

print("LSTM Output Shape:", output.shape) # [3, 5, 20]
print("Hidden State Shape:", hidden_state.shape) # [2, 3, 20]
```

```
LSTM Output Shape: torch.Size([3, 5, 20])
Hidden State Shape: torch.Size([2, 3, 20])
```

```
# ============================================================================
# EXPERIMENT 7: Object Detection using YOLO and RCNN
# QUESTION/OBJECTIVE: How do you develop and utilize object detection
# models using deep learning techniques to find objects in images?
# Note: Run '!pip install ultralytics' in a Colab cell before this.
# ============================================================================
!pip install ultralytics
from ultralytics import YOLO
import torch
from PIL import Image

# Load a pre-trained YOLOv8 nano model (downloads automatically)
model = YOLO('yolov8n.pt')

# Create a dummy image (black square) and save it
img = Image.new('RGB', (640, 640), color = 'black')
img.save('dummy.jpg')

# Run object detection inference
results = model('dummy.jpg')

# Print bounding boxes (will be empty for a black image, but proves it runs)
print("Detected boxes:", results[0].boxes.xyxy)
```

```
Collecting ultralytics
  Downloading ultralytics-8.4.33-py3-none-any.whl.metadata (39 kB)
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.2
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (3
Requirement already satisfied: opencv-python>=4.6.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (11.3.
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (6.0.3
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.
Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.16.3
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.10.0
Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (5.9.5
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.35
Collecting ultralytics-thop>=2.0.18 (from ultralytics)
  Downloading ultralytics_thop-2.0.18-py3-none-any.whl.metadata (14 kB)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->u
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.
```

```
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.12/dist-packages (from polars>
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ul
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytic
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch>=
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralyt
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultra
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ult
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultra
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->ma
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3-
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=1.
Downloading ultralytics-8.4.33-py3-none-any.whl (1.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/1.2 MB 21.1 MB/s eta 0:00:00
Downloading ultralytics_thop-2.0.18-py3-none-any.whl (28 kB)
Installing collected packages: ultralytics-thop, ultralytics
Successfully installed ultralytics-8.4.33 ultralytics-thop-2.0.18
Creating new Ultralytics Settings v0.0.6 file ✅
View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://d
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt to 'yolov8n.pt': 100% ━━━━

image 1/1 /content/dummy.jpg: 640x640 (no detections), 328.8ms
Speed: 6.1ms preprocess, 328.8ms inference, 9.2ms postprocess per image at shape (1, 3, 640, 640)
Detected boxes: tensor([], size=(0, 4))
```

```
# ======================================================================
# EXPERIMENT 8: Semantic and Instance Segmentation
# QUESTION/OBJECTIVE: How do you perform semantic and instance
# segmentation to classify images at the pixel level?
# ======================================================================
import torch
import torchvision.models.segmentation as segmentation

# Load a pre-trained Fully Convolutional Network (FCN) with ResNet50 backbone
model = segmentation.fcn_resnet50(weights=segmentation.FCN_ResNet50_Weights.DEFAULT)
model.eval() # Set to evaluation mode

# Dummy image: Batch=1, Channels=3, 256x256 pixels
dummy_image = torch.randn(1, 3, 256, 256)

with torch.no_grad():
    output = model(dummy_image)['out']

print("Segmentation Map Shape:", output.shape)
# [1, 21, 256, 256] -> 21 classes (PASCAL VOC), 256x256 resolution
```

```
Downloading: "https://download.pytorch.org/models/fcn_resnet50_coco-1167a1af.pth" to /root/.cache/torch/hub/check
100%|██████████| 135M/135M [00:00<00:00, 147MB/s]
Segmentation Map Shape: torch.Size([1, 21, 256, 256])
```

```
# ======================================================================
# EXPERIMENT 9: Autoencoders and Generative Adversarial Networks (GANs)
# QUESTION/OBJECTIVE: How do you construct generative architectures
# like autoencoders to compress and reconstruct data?
# ======================================================================
import torch
import torch.nn as nn

class Autoencoder(nn.Module):
    def __init__(self):
        super(Autoencoder, self).__init__()
        # Encoder: 784 (e.g., 28x28 image) -> 128 -> 64 (bottleneck)
        self.encoder = nn.Sequential(
            nn.Linear(784, 128),
            nn.ReLU(),
            nn.Linear(128, 64)
        )
        # Decoder: 64 -> 128 -> 784 (reconstruction)
        self.decoder = nn.Sequential(
            nn.Linear(64, 128),
            nn.ReLU(),
            nn.Linear(128, 784),
            nn.Sigmoid() # Output between 0 and 1
        )

    def forward(self, x):
        latent = self.encoder(x)
        reconstructed = self.decoder(latent)
```

```
        return reconstructed

model = Autoencoder()
dummy_flat_image = torch.randn(1, 784)
output = model(dummy_flat_image)
print("Autoencoder Output Shape:", output.shape)
```

```
# ======================================================================
# EXPERIMENT 10: Attention Mechanism and Encoder-Decoder Models
# QUESTION/OBJECTIVE: How do you apply attention-based mechanisms
# (Multihead Attention) to solve practical sequence-to-sequence problems?
# ======================================================================
import torch
import torch.nn as nn

# MultiheadAttention requires embed_dim and num_heads
embed_dim = 256
num_heads = 8
attention = nn.MultiheadAttention(embed_dim=embed_dim, num_heads=num_heads, batch_first=True)

# Dummy inputs for Query, Key, Value
# Batch=2, Sequence Length=10, Embedding Dimension=256
query = torch.randn(2, 10, embed_dim)
key = torch.randn(2, 10, embed_dim)
value = torch.randn(2, 10, embed_dim)

# Forward pass calculates attention weights and the context output
attn_output, attn_output_weights = attention(query, key, value)

print("Attention Output Shape:", attn_output.shape) # [2, 10, 256]
print("Attention Weights Shape:", attn_output_weights.shape) # [2, 10, 10]
```